Exercício em duplas:

Para cada trecho: **(a)** o que é impresso ou o que acontece? **(b)** qual regra explica?

1 · Java

```
static int f() {
    try {
        return 1;
    } finally {
        return 2;
    }
}
System.out.println(f());
```

(a) o que é impresso ou o que acontece?

R: 2

(b) qual regra explica?

R: Em Java, o bloco “finally” sempre executa, independentemente do resultado acima. Além disso, o return dele sempre se sobrepõe sobre qualquer outro return do try/catch.

2 · Python

```
def f(x):
    try:
        r = 10 / x
    except ZeroDivisionError:
        print("A")
        return -1
    else:
        print("B")
```

```
        return r
    finally:
        print("C")

print(f(0))
print(f(5))
```

(a) o que é impresso ou o que acontece?
R: A, C, -1 e depois B, C, 2.0

(b) qual regra explica?
R: Em Python, o finally roda sempre antes do return no try ou do except, por isso em ambos os casos, entre o print e o valor retornado, aparece a letra ‘C’.

3 · Go

```
n, _ := strconv.Atoi("12a")
fmt.Println(n * 2)
```

(a) o que é impresso ou o que acontece?
R: 0

(b) qual regra explica?
R: Em Go, usar o “_” faz com que o erro seja ignorado e o programa continue, mesmo com o valor errado.

4 · C++

```
try {
    throw 42;
} catch (double d) {
    std::cout << "double " << d;
} catch (...) {
    std::cout << "outro";
}
```

(a) o que é impresso ou o que acontece?
R: outro

(b) qual regra explica?
R: Em C++, a vinculação do tratamento de erros se dá pelo tipo. Um tipo int não casa com o tipo double, logo, cai no catch(...).

5 · Rust

```
let texto = "12a";
let n: i32 = texto.parse().unwrap();
println!("{}", n * 2);
```

(a) o que é impresso ou o que acontece?
R: Panic

(b) qual regra explica?
R: Em Rust, as exceções são classificadas em recuperáveis e irrecuperáveis. Em casos de erros irrecuperáveis, o Rust utiliza o mecanismo do Pânico.

6 · JavaScript

```
async function f() {
  throw new Error("falhou");
}
try {
  f();
} catch (e) {
  console.log("capturado");
}
console.log("fim");
```

(a) o que é impresso ou o que acontece?
R: Fim e depois soltará o erro de exceção não tratada.

(b) qual regra explica?
R: Em JavaScript, todo try sempre retorna uma Promise, e uma função async sempre retorna a

exceção de forma assíncrona. Dessa forma, a promise é rejeitada, mas não é capturada em tempo pelo try/catch, o programa finaliza e essa promise que ficou perdida estoura como erro.